

Track C — How to Run Guide

Track C is the **efficiency-focused evaluation track**. It takes a trained Track B checkpoint and applies various pruning/subsampling strategies to reduce inference latency while measuring the accuracy-speed trade-off.

Quick Start

```
cd d:\Thesis\Ego4d-LiteSTA\local_extraction

# Basic run (uses config defaults)
python -m trackC.trackC_pruning

# With CLIP re-ranking enabled
python -m trackC.trackC_pruning --clip on

# Baseline (no pruning)
python -m trackC.trackC_pruning --no_pruning
```

All CLI Options

```
python -m trackC.trackC_pruning --help
```

Argument	Values	Description
--config	trackC	Config file to load (from configs/)
--video_backbone	resnet18 , videomae_ego	Override backbone type
--checkpoint	path	Specific Track B checkpoint to evaluate
--rgtp	on , off	Enable/disable RGTP candidate pruning
--rgtp_rate	0.0-0.95	RGTP candidate pruning rate
--no_pruning	flag	Disable all pruning (same as --rgtp off)
--hotspot	on , off	Enable/disable hotspot priors
--clip	on , off	Enable/disable CLIP re-ranking
--frame_subsample	on , off	Enable frame subsampling (before fusion)
--keep_frames	2, 4, 8, 16	Frames to keep when subsampling

Argument	Values	Description
<code>--token_prune</code>	<code>on</code> , <code>off</code>	Enable token pruning before fusion
<code>--token_prune_rate</code>	0.0-0.95	Token pruning rate

New Efficiency Features (December 2024)

The Problem with Original RGTP

The original RGTP (Rollout-Guided Token Pruning) only prunes **candidates before the head**. However, latency analysis revealed:

Component	Latency	Percentage
-----	-----	-----
Backbone + FGTP	~35ms	99%
Head	~0.3ms	1%

RGTP prunes at the wrong stage! Pruning 50% of candidates only saves ~0.15ms.

Solution: Two New Efficiency Options

We added two features that prune **before** the expensive fusion stage:

1. Temporal Frame Subsampling

Reduces the number of video frames processed by FGTP. Since FGTP cost scales linearly with T (number of frames), this provides real speedup.

Config (`configs/trackC.yaml`):

```
efficiency:
  frame_subsample:
    enabled: true
    keep_frames: 4      # Keep 4 out of 16 frames → ~4x faster FGTP
    strategy: "uniform" # uniform, first, last, motion
```

CLI:

```
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 4
```

Strategies:

Strategy	Description
uniform	Evenly spaced frames (recommended)
first	First N frames
last	Last N frames (closest to action)
motion	Highest motion energy frames (expensive to compute)

2. Token Pruning Before Fusion

Prunes spatial tokens **before** FGTP + Dual Cross-Attention. Since attention has $O(N^2)$ complexity, reducing N provides quadratic speedup.

Config (`configs/trackC.yaml`):

```
efficiency:
  token_prune_before_fusion:
    enabled: true
    rate: 0.3           # Drop 30% of tokens → ~2x faster fusion
    min_keep: 49       # Keep at least 7x7 tokens
    strategy: "magnitude" # magnitude, variance, attention, random
```

CLI:

```
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.3
```

Strategies:

Strategy	Description
magnitude	L2 norm of token features (fast, default)
variance	Variance across channels (captures diversity)
attention	Dot-product with mean token (more expensive)
random	Random pruning (ablation baseline)

Example Commands

1. Baseline Evaluation (No Efficiency)

```
python -m trackC.trackC_pruning --rgtp off --clip on  
# or equivalently:  
python -m trackC.trackC_pruning --no_pruning --clip on
```

2. RGTP Only (Original Approach - Minimal Speedup)

```
python -m trackC.trackC_pruning --rgtp on --rgtp_rate 0.3 --clip on
```

3. Frame Subsampling Only (4 frames)

```
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 4 --clip on
```

4. Token Pruning Only (30% drop)

```
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.3 --clip  
on
```

5. Both New Features Combined (Maximum Speedup)

```
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 4 --  
token_prune on --token_prune_rate 0.3 --clip on
```

6. Aggressive Efficiency (for speed benchmarking)

```
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 2 --  
token_prune on --token_prune_rate 0.5 --rgtp off
```

7. All Features Enabled

```
python -m trackC.trackC_pruning --rgtp on --rgtp_rate 0.3 --frame_subsample on
--keep_frames 4 --token_prune on --token_prune_rate 0.3 --clip on
```

Expected Speedups

Configuration	FGTP Speedup	Total Speedup	Accuracy Impact
Baseline	1x	1x	100%
Frame subsample (8 frames)	~2x	~1.8x	Small drop
Frame subsample (4 frames)	~4x	~3x	Moderate drop
Token prune (rate=0.3)	~2x	~1.8x	Small drop
Token prune (rate=0.5)	~4x	~3x	Moderate drop
Both (4 frames + 0.3 rate)	~6-8x	~4-5x	Moderate drop

Configuration File Reference

The full efficiency section in `configs/trackC.yaml` :

```
# -----
-
# Efficiency Configuration (Track C Real Speedup)
# -----
-
efficiency:
  # --- Temporal Frame Subsampling ---
  frame_subsample:
    enabled: false
    keep_frames: 4          # 2, 4, 8, 16
    strategy: "uniform"    # uniform, first, last, motion

  # --- Token Pruning Before Fusion ---
  token_prune_before_fusion:
    enabled: false
    rate: 0.3              # 0.0-0.95
    min_keep: 49           # Minimum tokens to keep
    strategy: "magnitude"  # magnitude, variance, attention, random
```

Output Files

Metrics are saved to:

```
local_extraction/runs/Track_C/metrics/trackC_val_rateXX_YYYYMMDD_HHMMSS.json
```

Each JSON includes:

- All accuracy metrics (mAP, N_top5_mAP, All_top5_mAP, etc.)
- Latency measurements (mean, median, p90, p95)
- Efficiency config used:
 - `efficiency_frame_subsample_enabled`
 - `efficiency_frame_subsample_keep_frames`
 - `efficiency_token_prune_enabled`
 - `efficiency_token_prune_rate`

Ablation Study Commands

Run a Pareto curve comparing different efficiency settings:

```
# Baseline
python -m trackC.trackC_pruning --no_pruning --clip on

# Frame subsampling sweep
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 16 --clip on
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 8 --clip on
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 4 --clip on
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 2 --clip on

# Token pruning sweep
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.1 --clip on
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.2 --clip on
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.3 --clip on
python -m trackC.trackC_pruning --token_prune on --token_prune_rate 0.5 --clip on

# Combined (best settings)
python -m trackC.trackC_pruning --frame_subsample on --keep_frames 4 --token_prune on --token_prune_rate 0.3 --clip on
```

Troubleshooting

"Checkpoint not found"

```
# List available checkpoints
dir local_extraction\runs\Track_B\checkpoints\*.pt

# Use a specific checkpoint
python -m trackC.trackC_pruning --checkpoint
"local_extraction/runs/Track_B/checkpoints/trackB_best_mAP_0.3708_20251225_224
220.pt"
```

"No valid samples processed"

- Check that Track A Stage B has been run
- Verify validation manifest exists in `runs/Track_A/*/head_val*.json`

Low speedup despite efficiency settings

- Ensure `enabled: true` is set in config or use CLI flags
- Check output JSON for `efficiency_*` fields to confirm settings were applied

Summary of Changes (December 2024)

Files Modified

1. `configs/trackC.yaml`
 - Added `efficiency.frame_subsample` section
 - Added `efficiency.token_prune_before_fusion` section
 - Added documentation comments explaining each option
2. `trackC/trackC_pruning.py`
 - Added `FrameSubsampleConfig` dataclass
 - Added `TokenPruneBeforeFusionConfig` dataclass
 - Added `EfficiencyConfig` dataclass
 - Added `_subsample_frames()` function
 - Added `_prune_tokens_before_fusion()` function
 - Added CLI arguments: `--rgtp`, `--frame_subsample`, `--keep_frames`, `--token_prune`, `--token_prune_rate`
 - Integrated efficiency features in main evaluation loop
 - Added efficiency metrics to output JSON

Why These Changes?

The original RGTP approach pruned candidates **after** the fusion stage, but the fusion (FGTP + Dual Cross-Attention) accounts for 99% of inference latency. The new features prune **before** fusion:

BEFORE (RGTP):

Backbone → FGTP (35ms) → ROI Pool → RGTP prune → Head (0.3ms)

Pruning here saves <1%

AFTER (New Efficiency):

Backbone → Subsample → Token Prune → FGTP (reduced) → ROI Pool → Head

4x speedup

2-4x speedup